CSA0623 DAA
LAB EXPERIMENTS

NAME: V.KAVIYA
REG NO: 192421022

Experiment 1: Linear Search (Iterative)

main.py

```python
arr = [10, 25, 30, 45, 50]
key = 30

found = False

for i in range(len(arr)):
    if arr[i] == key:
        print("Key found at index", i)
        found = True
        break

if not found:
    print("Key not found")
```

Output

```
Key found at index 2

=== Code Execution Successful ===
```

Experiment 2: Binary Search (Recursive)

main.py

```python
def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)

    return -1


arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr)-1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Output

```
Key found at index 3

=== Code Execution Successful ===
```

Experiment 3: Factorial (Iterative vs Recursive)

main.py | Share | Run | Output

```
n = 5
fact = 1

for i in range(1, n + 1):
    fact *= i

print("Iterative Factorial:", fact)


# Recursive Factorial

def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print("Recursive Factorial:", factorial(n))
```

```
Iterative Factorial: 120
Recursive Factorial: 120

=== Code Execution Successful =
```

Experiment 4: Fibonacci Series

main.py | Share | Run | Output

```
n = 6

a = 0
b = 1

print("Iterative Fibonacci:")

for i in range(n):
    print(a, end=" ")
    c = a + b
    a = b
    b = c


# Recursive Fibonacci

print("\nRecursive Fibonacci:")

def fibonacci(num):
    if num <= 1:
        return num
    return fibonacci(num - 1) + fibonacci(num - 2)

for i in range(n):
    print(fibonacci(i), end=" ")
```

```
Iterative Fibonacci:
0 1 1 2 3 5
Recursive Fibonacci:
0 1 1 2 3 5
=== Code Execution Successful ===
```

Experiment 5: Matrix Multiplication

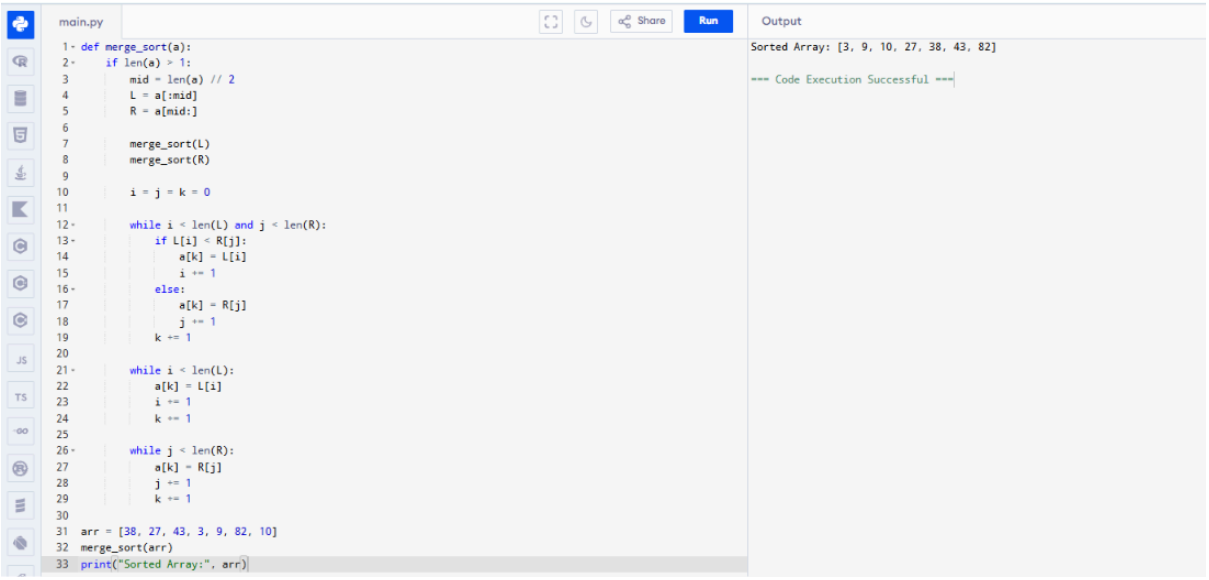
main.py

```python
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

result = [[0, 0],
          [0, 0]]

for i in range(2):
    for j in range(2):
        for k in range(2):
            result[i][j] += A[i][k] * B[k][j]

print("Result Matrix:")

for row in result:
    print(row)
```

Output

```
Result Matrix:
[19, 22]
[43, 50]

=== Code Execution Successful ===
```

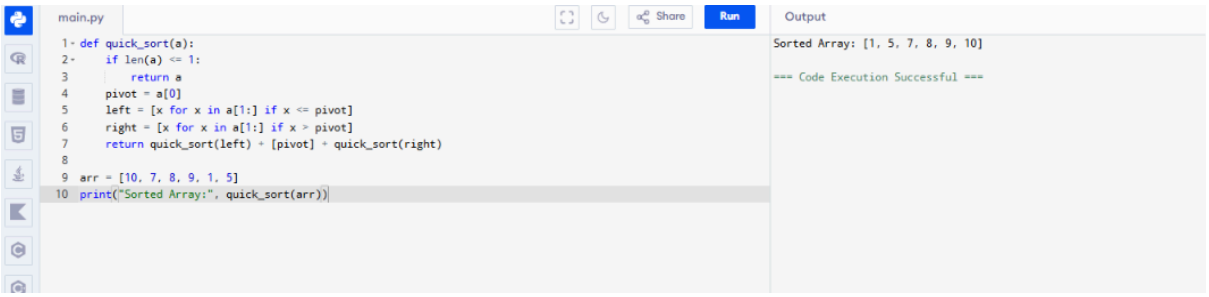
Experiment 6 : Merge Sort



```python
def merge_sort(a):
    if len(a) > 1:
        mid = len(a) // 2
        L = a[:mid]
        R = a[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                a[k] = L[i]
                i += 1
            else:
                a[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            a[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            a[k] = R[j]
            j += 1
            k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted Array:", arr)
```

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===
```

Experiment 7 : Quick Sort



```python
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

arr = [10, 7, 8, 9, 1, 5]
print("Sorted Array:", quick_sort(arr))
```

```
Sorted Array: [1, 5, 7, 8, 9, 10]

=== Code Execution Successful ===
```

Experiment 8 : Towers of Hanoi

```python
def hanoi(n, s, a, d):
    if n == 1:
        print(s, "->", d)
        return
    hanoi(n-1, s, d, a)
    print(s, "->", d)
    hanoi(n-1, a, s, d)

n = 3
hanoi(n, 'A', 'B', 'C')
```

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C

=== Code Execution Successful ===
```

Experiment 9 : GCD (Euclidean Algorithm)

```python
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

print("GCD =", gcd(48, 18))
```

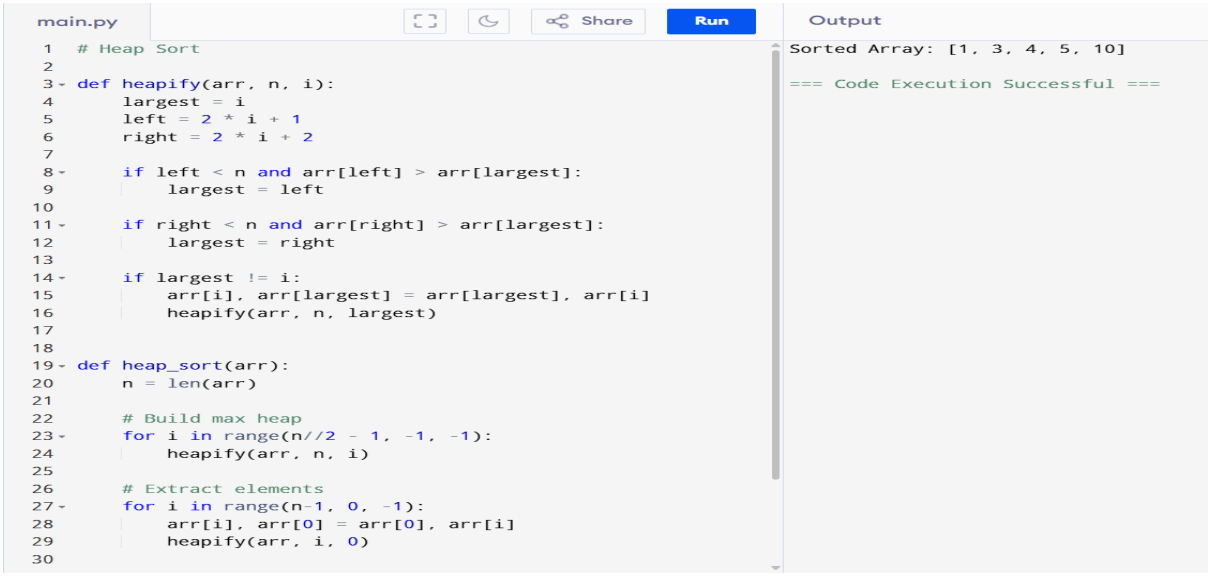
```
GCD = 6

=== Code Execution Successful ===
```

Experiment 10 : Insertion Sort

```python
arr = [12, 11, 13, 5, 6]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key

print("Sorted Array:", arr)
```

```
Sorted Array: [5, 6, 11, 12, 13]

=== Code Execution Successful ===
```

Experiment 11: Heap Sort



```python
# Heap Sort

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    # Build max heap
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

```

```
Sorted Array: [1, 3, 4, 5, 10]

=== Code Execution Successful ===
```

Experiment 12: Counting Inversions

main.py | Share | Run | Output

```python
# Count Inversions (Brute Force)

arr = [2, 4, 1, 3, 5]
count = 0

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            count += 1

print("Number of Inversions:", count)
```

```
Number of Inversions: 3

=== Code Execution Successful ===
```

Experiment 13: Maximum Subarray Sum

main.py | Share | Run | Output

```python
# Maximum Subarray Sum

arr = [-2, -3, 4, -1, -2, 1, 5, -3]

max_sum = arr[0]
current_sum = arr[0]

for i in range(1, len(arr)):
    current_sum = max(arr[i], current_sum + arr[i])
    max_sum = max(max_sum, current_sum)

print("Maximum Subarray Sum:", max_sum)
```

```
Maximum Subarray Sum: 7

=== Code Execution Successful ===
```

Experiment 14: Exponentiation

main.py | Share | Run | Output

```python
x = 2
n = 10

result = 1

for i in range(n):
    result *= x

print("Iterative Power:", result)


# Recursive Fast Power

def power(x, n):
    if n == 0:
        return 1

    half = power(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

print("Recursive Fast Power:", power(x, n))
```

```
Iterative Power: 1024
Recursive Fast Power: 1024

=== Code Execution Successful ===
```

Experiment 15: Closest Pair of Points

main.py

```python
import math
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
min_dist = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

        if dist < min_dist:
            min_dist = dist
            pair = (points[i], points[j])

print("Closest Pair:", pair)
print("Distance:", min_dist)
```

Output

```
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

=== Code Execution Successful ===
```

Experiment 16: First palindromic string

main.py

```python
words = ["abc","car","ada","racecar","cool"]

for w in words:
    if w == w[::-1]:
        print(w)
        break
else:
    print("")
```

Output

```
ada

=== Code Execution Successful ===
```

Experiment 17: Find array values

main.py

```python
nums1 = [2,3,2]
nums2 = [1,2]

a1 = sum(1 for x in nums1 if x in nums2)
a2 = sum(1 for x in nums2 if x in nums1)

print([a1, a2])
```

Output

```
[2, 1]

=== Code Execution Successful ===
```

Experiment 18: Sum of squares of distinct counts

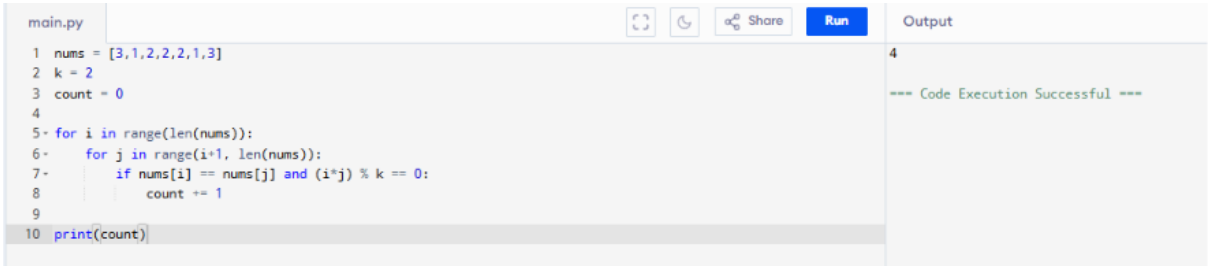
main.py

```python
nums = [1,2,1]
ans = 0

for i in range(len(nums)):
    s = set()
    for j in range(i, len(nums)):
        s.add(nums[j])
        ans += len(s) ** 2

print(ans)
```

Output

```
15

=== Code Execution Successful ===
```

Experiment 19: Count equal and divisible pairs



Experiment 20: Find Maximum Element (Least Time Complexity - O(n))

TEST CASE 1

TEST CASE 2

TEST CASE 3

Experiment 21: Sort the List and Find Maximum Element

TEST CASE 1

main.py

```
arr = []
```

Output

```
=== Code Execution Successful ===
```

Experiment 22: Find Unique Elements

main.py

```
arr = [3,7,3,5,2,5,9,2]

unique = []

for i in arr:
    if i not in unique:
        unique.append(i)

print(unique)
```

Output

```
[3, 7, 5, 2, 9]

=== Code Execution Successful ===
```

Experiment 23: Bubble Sort

main.py

```
arr = [64,34,25,12,22,11,90]

for i in range(len(arr)):
    for j in range(len(arr)-1-i):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]

print("Sorted Array:", arr)
```

Output

```
Sorted Array: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===
```

Experiment 24: Binary Search

main.py

```
arr = [3,4,6,-9,10,8,9,30]
key = 10

arr.sort()

low = 0
high = len(arr)-1
found = False

while low <= high:
    mid = (low+high)//2

    if arr[mid] == key:
        print("Element", key, "is found at position", mid+1)
        found = True
        break
    elif arr[mid] < key:
        low = mid+1
    else:
        high = mid-1

if not found:
    print("Element", key, "is not found")
```

Output

```
Element 10 is found at position 7

=== Code Execution Successful ===
```

Experiment 25: Merge Sort (Without Built-in Sort)

main.py

```python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

arr = [5,2,3,1]
merge_sort(arr)
print(arr)
```

Output

```
[1, 2, 3, 5]

=== Code Execution Successful ===
```